

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE Nº 09

NOMBRE COMPLETO: Luis Daniel Salazar Islas

Nº de Cuenta: 320335163

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 28/10/2025

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Actividad 1

Se requiere cambiar la velocidad del número cambiante que itera sobre la textura de los números.

Para ello establecí que la velocidad del cambio de textura pasé de un ciclo de reloj a 1.5 s.

Declaro una variable para guardar el tiempo actual en forma de segundo a través de la función glfwGetTime()

```
GLfloat tiempoActual;
```

Declaro otra para guardar la última actualización

```
float ultimoCambioNumero = 0.0f;
```

Al obtener el delta verifico si es mayor a 1.5, si lo es se realiza la actualización de la variable

```
tiempoActual = glfwGetTime();
if (tiempoActual - ultimoCambioNumero >= 1.5f)
{
    toffsetnumerocambiau += 0.25f;
    if (toffsetnumerocambiau >= 1.0f)
        toffsetnumerocambiau = 0.0f;
    ultimoCambioNumero = tiempoActual; // Guardar el tiempo actual
}
```

Ejecución

Cambio de 4 a 1





Actividad 2

Para el siguiente ejercicio se requiere alternar entre la textura del número 1 y 2 a una velocidad visible para el ser humano. Se sigue una lógica parecida al anterior ejercicio, se establecen dos variables para calcular los deltas de tiempo. Aunque agregamos una bandera para alternar entre los números

Declaración de variables

```
bool usarNumero1 = true;
float ultimoCambioTextura = 0.0f;
```

```
GLfloat tiempoActualTextura;
```

Condicional para escoger la textura

```
tiempoActualTextura = glfwGetTime();
if (tiempoActualTextura - ultimoCambioTextura >= 1.0f)
{
    usarNumero1 = !usarNumero1; // Alternar entre true y false
    ultimoCambioTextura = tiempoActualTextura;
}
```

Se usa la textura correspondiente

```
if (usarNumero1)
    Numero1Texture.UseTexture();
else
    Numero2Texture.UseTexture();
```

Ejecución

Textura 1



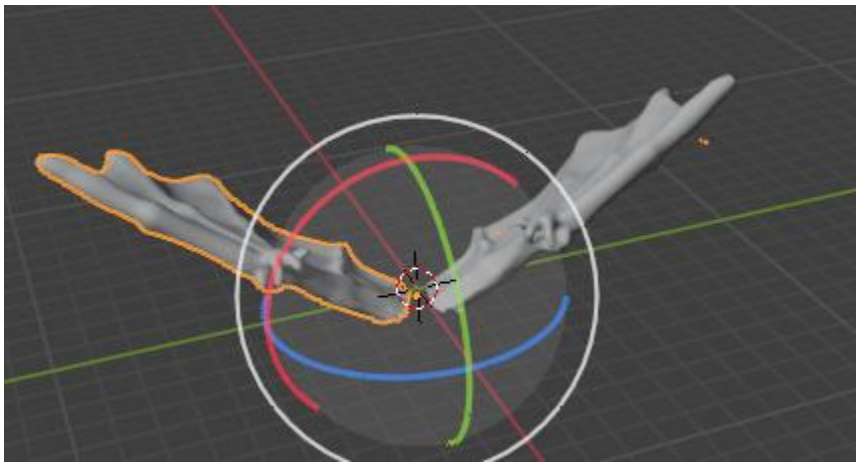
Textura 2



Actividad 3

Para la última actividad se requiere animar a un dragón al dotarle movimiento a sus alas, hacer que se mueva y que regrese en un ciclo.

Para animar las alas por supuesto se tuvo que separarlas desde blender y exportarlas como modelos diferentes para después con jerarquía unirlas al dragón.



Al separar las alas se exporto al dragón sin ellas. Se le agrega una bandera para indicar si esta avanzando hacia adelante o atrás, de forma que al cambiar de curso gire su orientación.

```
//dragonavance para moverlo hacia adelante y atrás en X
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(dragonavance, 5.0f+3*sin(glm::radians(angulovaria)), 6.0));
if (!dragonAvanza) {
    model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
}
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
modelaux = model;
//model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
/*color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//Dragon_M.RenderModel();
Dragon_no_wings.RenderModel();
```

Posteriormente al exportar las alas, se colocó que rotaran para simular el efecto de aleteo.Para que los aleteos estuvieran sincronizados con la marcha del dragón se usó la misma variable senoidal con la que se mueve.

```
model = modelaux;
model = glm::rotate(model,glm::radians(35-35*sin(glm::radians(angulovaria))),glm::vec3(1.0f,0.0f,0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
/*color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
rightWing.RenderModel();

model = modelaux;
model = glm::rotate(model, glm::radians(35 - 35 * sin(glm::radians(angulovaria))), glm::vec3(-1.0f, 0.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
/*color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
leftWing.RenderModel();
```

Para delimitar la cantidad de movimiento del dragón se usa una lógica similar a los anteriores ejercicios

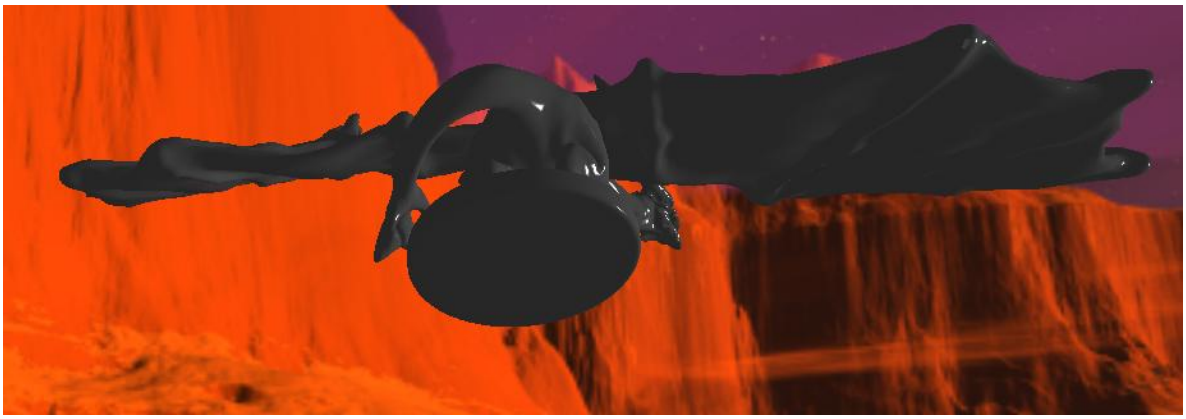
```
//animación del dragón
if (dragonAvanza) {
    if (dragonavance > -20.0f) {
        dragonavance -= avanceoffset * deltaTime;
    }
    else {
        dragonAvanza = !dragonAvanza;
    }
}
else {
    if (dragonavance < 0.0f) {
        dragonavance += avanceoffset * deltaTime;
    }
    else {
        dragonAvanza = !dragonAvanza;
    }
}
```

Ejecución

De frente



De espaldas



2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código

Para realizar las dos primeras actividades no hubo mucha dificultad, solo fue meditar un poco acerca de la manera más sencilla para detectar el paso del tiempo, lo cual no requirió de mucha búsqueda gracias a que ya existía una función de glfw.

La última actividad fue un poco más demandante pues tuve que considerar más casos como la sincronización del aleteo con el movimiento para que se viera natural el movimiento, también la acción de rotar dependiendo de la bandera. Aún así, no supuso un gran problema implementar todas estas características.

3. Conclusión:
    a. Los ejercicios de la clase

La complejidad de los ejercicios de clase fue adecuada, no se tenía que hacer muchos cambios, en cambio lo más importante era entender el flujo y en que parte del código era conveniente agregar una modificación, aún así, no siento que el ejercicio haya sido complejo.

    b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias.

La explicación me pareció correcta, en el pasado ya hemos trabajado con ese proceso de rotar, trasladar y escalar por medio de teclado, por lo que únicamente tuvimos que aprender a automatizar lo ya visto. Por lo tanto, no siento que se requiriera una explicación muy profunda para los conceptos básicos de la animación.